TFGUARD: A Machine Learning–Assisted Framework for Security Verification of Infrastructure-as-Code

[Your Full Name], [Team Member 2], [Team Member 3], [Team Member 4], [Supervisor Name]
[Department / School], [University], [Country]
[your.email@example.com], [member2@example.com], [member3@example.com], [member4@example.com], [supervisor@example.com]

1. INTRODUCTION

Infrastructure-as-Code (IaC) is the practice of writing cloud infrastructure as text files instead of setting it up by hand. Terraform is the most widely used IaC language. It makes deployments repeatable, but it also means that one wrong line in a file becomes a real security hole as soon as it is applied. Common examples are a storage bucket that anyone on the internet can read, a firewall rule that opens SSH to `0.0.0.0/0`, and a database created without encryption. Because the same file is often reused across many environments, one mistake can be repeated many times.

Research shows these mistakes are common. Rahman et al. studied thousands of IaC scripts and found seven types of security problems that appear again and again, such as hard-coded passwords and the use of insecure protocols [1, 12]. A more recent study of 812 open-source Terraform projects found that encryption at rest is the security setting developers most often leave out, across all major cloud providers [3].

The usual defence is a rule-based scanner such as Checkov [7], Trivy [8], KICS or Terrascan. These tools compare the code against a list of known bad patterns. They are fast and easy to use, but they have three limits. First, they can only find problems somebody has already written a rule for, so a new or unusual mistake is missed completely. Second, they report many false alarms, because a rule looks at the pattern and not at the situation around it. Developers then stop paying attention to the warnings. Third, they cannot judge how serious a problem is: a public bucket holding test data and one holding customer data look the same to a rule.

Machine learning can help here because a model can look at many properties of a resource together, rather than checking one condition at a time, and can give a risk score instead of a simple yes or no. This project builds **TFGUARD**. It reads each Terraform `resource` block, turns it into a list of numbers describing its structure and its security settings, and uses an XGBoost classifier [5] to predict how likely that block is to be insecure. The model runs after the rule scanner, not instead of it. The scanner results are added as extra inputs to the model, and the model gives a score to every resource, including the ones the scanner did not flag. This lets the system rank the real problems first, filter out likely false alarms, and also point out risky resources that no rule covers. Each score comes with a SHAP explanation [6] that lists the settings which caused it, so the user can see the reason behind the score.

The main research question is not whether the model can copy the scanner's output. It is whether the model can find problems it was never trained on. To test this, one whole type of misconfiguration is removed from the training data, and the model is then checked to see if it still detects that type. This test, together with a hand-checked test set and a set of deliberately weakened files, is what makes the project a study rather than only a tool.

2. PROJECT SCOPE

The scope of this project is to design, build and evaluate TFGUARD, a system that combines rule-based scanning with machine learning to score the security risk of Terraform code before it is deployed. The system consists of the following parts:

- **Code Normalisation:** A parser built on `python-hcl2` that reads Terraform files and converts them into one common structure. This follows the approach used by GLITCH, which analyses several IaC languages through a shared representation [2, 10]. This project evaluates Terraform on AWS only, but the interface is kept general so that Kubernetes YAML can be added later as future work.
- **Rule-Based Layer:** Two scanners, Checkov and Trivy, are run over the code. Their results serve three purposes: they are the baseline to compare against, they provide the labels used to train the model, and the cases where the two scanners disagree are studied separately.
- **Feature Extraction:** A tested module that turns each resource block into about 80 to 150 numeric features. These cover the structure of the block (resource type, number of settings, nesting depth), its security settings (encryption and key management, open IP ranges, exposed admin ports, wildcards in IAM policies, logging, public-access controls, and possible hard-coded secrets) and its context (tags, names that suggest sensitive data, provider and region, and how many other resources depend on it). This is the main technical part of the project and reuses ideas from existing Terraform metric tools [4].
- **Risk Scoring Model:** An XGBoost classifier [5], compared against simpler baselines (majority class, logistic regression and random forest), tuned automatically and trained with class weighting because insecure blocks are a small minority of the data.
- **Explainability:** A SHAP explanation [6] for every score, listing the features that pushed the score up or down, so that a user can check whether the model is reasoning about security or about something irrelevant.
- **Command-Line Tool:** A packaged tool that applies a threshold to the score, prints results as text, JSON or SARIF, and returns a pass or fail exit code so it can be used inside a CI/CD pipeline.
- **Evaluation Method:** A dataset of public Terraform code collected from GitHub, with duplicates removed and the source of every file recorded. Training and test data are split by repository, not at random, because Terraform is often copied between projects and a random split would give falsely high scores. Results are reported using precision, recall, F1, PR-AUC and MCC rather than accuracy alone.
- **Human Feedback:** When a security analyst marks a result as correct or incorrect, that decision is stored as new training data and is only used at the next planned retraining. The model never learns automatically from its own output.

The following are outside the scope of this project and are recorded as future work: comparing several cloud providers, scanning Kubernetes files, using large language models, and building a web dashboard.

3. CONTRIBUTIONS

This work makes the following key contributions:

1. **The TFGUARD Framework:** A design that places a machine learning model on top of existing rule scanners instead of replacing them, so the system keeps the reliability of known rules while adding risk scoring and context awareness, and

produces a tool that can run in a CI/CD pipeline.

2. **A Feature Set for Terraform Security:** A documented and tested module that converts a Terraform resource block and its scanner results into a fixed list of numbers. It will be released so that other researchers can reuse it instead of writing their own Terraform parser.
3. **A Test for Generalisation:** An experiment in which one whole class of misconfiguration is removed from the training labels and the model is then tested on it. This measures whether the model has learned something about security or has only copied the scanner. It is supported by a second test using files that were deliberately weakened. Together these address the risk of circular reasoning, which earlier work in this area has not tested.
4. **Explainable Risk Ranking:** A reporting method that turns each score into a short list of reasons, so developers are told not only what is risky but also why, and in what order to fix it. This targets the false-alarm problem directly.
5. **A Reliable Dataset:** A collection of public Terraform resource blocks with duplicates removed, split by repository, with automatic labels, hand-checked labels and synthetic labels, and the source of each file recorded. Earlier machine learning work on Terraform has used only a few hundred hand-picked samples and has not tested generalisation [11].
6. **Evaluation Against Real Tools:** A direct comparison between the model, the two rule scanners, and combinations of the two on a hand-checked test set, measuring the change in the false alarm rate and showing whether the approach is worth using in practice.

4. KEY THEORIES

1. **Gradient Boosted Decision Trees.** XGBoost builds many small decision trees one after another, where each new tree corrects the errors of the trees before it, with limits on tree size to avoid overfitting [5]. This suits the project because the data is a table of mixed numeric and categorical features, and the results can still be traced back to individual features.
2. **Shapley Values from Game Theory.** SHAP explains a single prediction by fairly dividing it among the input features, using Shapley values from cooperative game theory [6]. This is the theory behind the explanation attached to every finding.
3. **Static Program Analysis.** The parsing stage is based on static analysis: examining source code without running it, using a structured representation instead of raw text. Converting several IaC languages into one shared representation before analysis is the idea introduced by GLITCH [2, 10], and it is what allows the design to work for more than one language.
4. **Learning with Imbalanced Data.** Insecure blocks are only a small share of all blocks, and Terraform code is often copied between projects. Statistical learning theory requires the training and test data to be independent, which is why the data is split by repository, duplicates are removed, and PR-AUC is used instead of accuracy.
5. **Security Smells and Shift-Left Security.** The list of recurring IaC security smells [1, 12] defines what the model is trying to detect. The shift-left principle, which says security checks should happen as early as possible, is the reason the tool runs at commit time rather than after deployment.
6. **Human-in-the-Loop Learning.** Analyst decisions are stored and applied only at a planned retraining, rather than fed back automatically. This follows the standard rule that a model which trains on its own predictions will drift and reinforce its own mistakes.

REFERENCES

[1] A. Rahman, C. Parnin, and L. Williams, "The Seven Sins: Security Smells in Infrastructure as Code Scripts," in *Proc. 41st Int. Conf. Software Engineering (ICSE)*, Montreal, Canada, 2019, pp. 164–175. doi: 10.1109/ICSE.2019.00033

[2] N. Saavedra and J. F. Ferreira, "GLITCH: Automated Polyglot Security Smell Detection in Infrastructure as Code," in *Proc. 37th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, Rochester, MI, USA, 2022. doi: 10.1145/3551349.3556945. arXiv:2205.14371

[3] A. Verdet, M. Hamdaqa, L. Da Silva, and F. Khomh, "Assessing the adoption of security policies by developers in Terraform across different cloud providers," *Empirical Software Engineering*, vol. 30, art. 74, 2025. doi: 10.1007/s10664-024-10610-0

[4] M. Begoug, M. Chouchen, and A. Ouni, "TerraMetrics: An Open Source Tool for Infrastructure-as-Code (IaC) Quality Metrics in Terraform," in *Proc. 32nd IEEE/ACM Int. Conf. Program Comprehension (ICPC)*, Lisbon, Portugal, 2024. doi: 10.1145/3643916.3644439

[5] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794. doi: 10.1145/2939672.2939785. arXiv:1603.02754

[6] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems (NeurIPS) 30*, Long Beach, CA, USA, 2017, pp. 4765–4774. arXiv:1705.07874

[7] Prisma Cloud (formerly Bridgecrew), "Checkov: Static code analysis for infrastructure as code," GitHub repository. [Online]. Available: https://github.com/bridgecrewio/checkov

[8] Aqua Security, "Trivy: Vulnerability, misconfiguration and secret scanner," GitHub repository. [Online]. Available: https://github.com/aquasecurity/trivy

[9] A. Rahman, R. Mahdavi-Hezaveh, and L. Williams, "A systematic mapping study of infrastructure as code research," *Information and Software Technology*, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0950584918302507. arXiv:1807.04872

[10] N. Saavedra, J. Gonçalves, M. Henriques, J. F. Ferreira, and A. Mendes, "Polyglot Code Smell Detection for Infrastructure as Code with GLITCH," in *Proc. 38th IEEE/ACM Int. Conf. Automated Software Engineering (ASE), Tool Demonstrations*, Luxembourg, 2023. arXiv:2308.09458

[11] "TerraSecure: AI-powered security scanner for Terraform and HCL," GitHub repository. [Online]. Available: https://github.com/JashwanthMU/TerraSecure

[12] A. Rahman, M. R. Rahman, C. Parnin, and L. Williams, "Security Smells in Ansible and Chef Scripts: A Replication Study," *ACM Trans. Software Engineering and Methodology*, vol. 30, no. 1, 2021. doi: 10.1145/3408897. arXiv:1907.07159